

Lecture Notes, Fast

A practical LaTeX + VimTeX desk reference

The 10-second loop

Type notes with snippets and the short formatting triggers. Let the 650 ms save debounce do its work. When you want a checkpoint: `Space la` saves and compiles; `Space lv` opens the PDF; `Space le` shows errors.

Your key language

Key	Action
<code>Space</code>	leader key for commands
<code>jk</code>	leave Insert mode quickly
<code>Space w</code>	save current file
<code>Space q</code>	quit window
<code>Ctrl-h/j/k/l</code>	move between splits
<code>Space zw</code>	distraction-free writing

Compile and inspect

Keys	Use
<code>Space ll</code>	start/refresh continuous compile
<code>Space la</code>	save, then compile
<code>Space lv</code>	view PDF in Skim
<code>Space le</code>	inspect compiler errors
<code>Space lt</code>	toggle table of contents
<code>Space lk</code>	stop compiler
<code>Space lm</code>	one-shot compile
<code>Space lo</code>	show compiler output
<code>Space lc</code>	clean auxiliary files
<code>Space li</code>	show VimTeX project info
<code>Space ts</code>	toggle spell checking

Quickfix does not open automatically.

Use `Space le` only when you want diagnostics, so your cursor and page stay stable while lecturing.

Find things without leaving the file

<code>Space ff</code>	find a file
<code>Space fg</code>	search text across the project
<code>Space fs</code>	jump through document symbols
<code>Space fb</code>	choose an open buffer
<code>Space e</code>	toggle the file tree

The fast lecture pattern

1. Open or create a ‘.tex’ file. A new file receives your template automatically.
2. Type a trigger such as `defn`, `thm`, `proof`, `al`, or `eq`; expand it with your completion key (usually `Tab`).
3. Use `;c` for a fresh bold Claim paragraph, `;p` for Proof, and `;r` for Remark.
4. Press `Space la` at a natural pause. If something breaks, press `Space le`.

Formatting while you type

One-line emphasis

In Insert mode:

<code>;b</code>	inserts <code>\textbf{}</code> and places the cursor inside.
<code>;i</code>	inserts <code>\emph{}</code> and places the cursor inside.
<code>;c</code>	starts a new paragraph labeled Claim .
<code>;p</code>	starts a new paragraph labeled Proof .
<code>;r</code>	starts a new paragraph labeled Remark .
<code>\(</code>	inserts <code>\(</code> with the cursor inside.
<code>\[</code>	inserts <code>\[</code> with the cursor inside.

LuaSnip triggers already installed

Statements and proof structure

prb	problem plus solution
prob	numbered problem
sol, ans	solution or answer block
defn	definition
thm, lem,	theorem, lemma, proposition, corollary
prop, cor	
ex, rem,	example, remark, proof
proof	

Display and layout

al, eq	align or equation display
cases, mat	cases or matrix
enum, item	numbered or bulleted list
sec, ssec	section or subsection

Mathematical shorthand

rr, cc, qq, zz, nn	$\mathbb{R}, \mathbb{C}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
abs, norm, ip,	absolute value, norm, inner product,
set	set
span, rank,	common linear/algebra operators
ker, im	
lim, int, pd,	limits, integrals, partial/ordinary
dv	derivatives
prob, cond, ev,	probability notation
var, cov	

Good source habits

- Put one mathematical claim per paragraph. Start it with `;c`.
- Use `align*` for multiple equalities; put ‘&’ before the alignment point and ‘
’ between lines.
- Use `\text{...}` inside math for words: $x > 0$ if $x \neq 0$.
- Label important results: `\label{thm:name}`; reference them with `\cref{thm:name}`.
- Keep scratch explanations in `\textbf{Claim.}`, `{Proof.}`, or `{Idea.}` rather than burying them in comments.

Useful emergency commands

<code>:VimtexCompile</code>	start/refresh compile
<code>:VimtexView</code>	open the PDF
<code>:VimtexErrors</code>	inspect errors
<code>:VimtexTocToggle</code>	show/hide outline
<code>:VimtexStop</code>	stop compilation
<code>:VimtexInfo</code>	inspect project configuration

Remember

If a shortcut is forgotten: press `Space` and wait for which-key/help prompts, or use `:h vimtex`.